Design using System Verilog

Assignment_1

- Toshiv Kumar, 1024060266, 3F32, ECE

Aim: Familiarization with SystemVerilog data types and arrays, including performing operations on fixed and dynamic arrays.

1. Write the SystemVerilog code to:

i. Declare a 2-state array, my_array, that holds four 12-bit values

ii. Initialize my_array so that:

my_array[0] = 12'h012

my_array[1] = 12’h345

my_array[2] = 12'h678

my_array[3] = 12’h9AB

iii. Traverse my_array and print out bits [5:4] of each 12-bit element

a. With a for loop

b. With a foreach loop

Code:

a. With a for loop

```
`timescale 1ns / 1ps



// Toshiv Kumar(NAME) , 1024060266(Roll Number)
module question_1; // In the form of testbench.
  bit [11:0] my_array [0:3];
  integer i;
  initial begin
   i = 0;
    my_array[0] = 12'h012;
```

```
    my_array[1] = 12'h345;
    my_array[2] = 12'h678;
    my_array[3] = 12'h9AB;

    for (i = 0; i <= 3; i = i+1) begin
      $display("my_array[%0d][5:4] = %b", i, my_array[i][5:4]);
    end
  end
endmodule
```

b. With a foreach loop

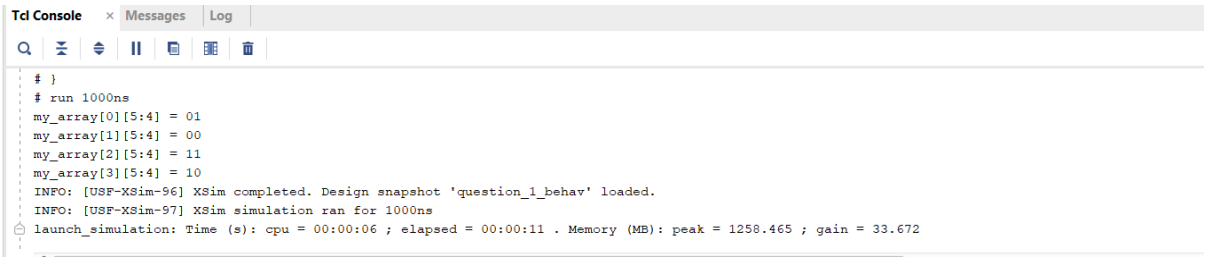
```
`timescale 1ns / 1ps



// Toshiv Kumar(NAME) , 1024060266(Roll Number)
module question_1; // In the form of testbench.
  bit [11:0] my_array [0:3];
  integer i;
  initial begin
   i = 0;
    my_array[0] = 12'h012;
    my_array[1] = 12'h345;
    my_array[2] = 12'h678;
    my_array[3] = 12'h9AB;

                    foreach (my_array[i]) begin
         $display("my_array[%0d][5:4] = %b", i, my_array[i][5:4]);
        end
  end
endmodule
```

Output/Screenshot:



2. Write the SystemVerilog code to:

i. Declare a 2-state two dimensional array, array_2d (4 rows and 3 columns), that holds 12 integer values.

ii. Initialize my_array so that: array_2d[0][0] = 0 array_2d[0][1] = 1 array_2d[0][2] = 2 array_2d[1][0] = 3

array_2d[1][1] = 4 array_2d[1][2] = 5 array_2d[2][0] = 6 array_2d[2][1] = 7 array_2d[2][2] = 8 array_2d[3][0] = 19 array_2d[3][1] = 20 array_2d[3][2] = 21

iii. Print out the values stored in array_2d and verify them with initialized values. Hint: use foreach loop.

Code:

`timescale 1ns / 1ps

// Toshiv Kumar(NAME) , 1024060266(Roll Number)

/*

`%3d` prints decimal in a **minimum width of 3 characters**.

integer is a 32-bit signed 4-state type in SystemVerilog.

int / int unsigned is a 32-bit 2-state type:

int = signed

int unsigned = unsigned

$display("%3d", 7);   // " 7"

$display("%3d", 45);   // " 45"

$display("%3d", 123); // "123"

It pads with spaces on the left if needed.

*/

module question_2(

  );

 int unsigned array_2d [0:3][0:2];

  integer i;

  integer j;

  initial begin

   i = 0;

        j = 0;

           array_2d[0][0] = 0;

    array_2d[0][1] = 1;

    array_2d[0][2] = 2;

    array_2d[1][0] = 3;

    array_2d[1][1] = 4;

    array_2d[1][2] = 5;

    array_2d[2][0] = 6;

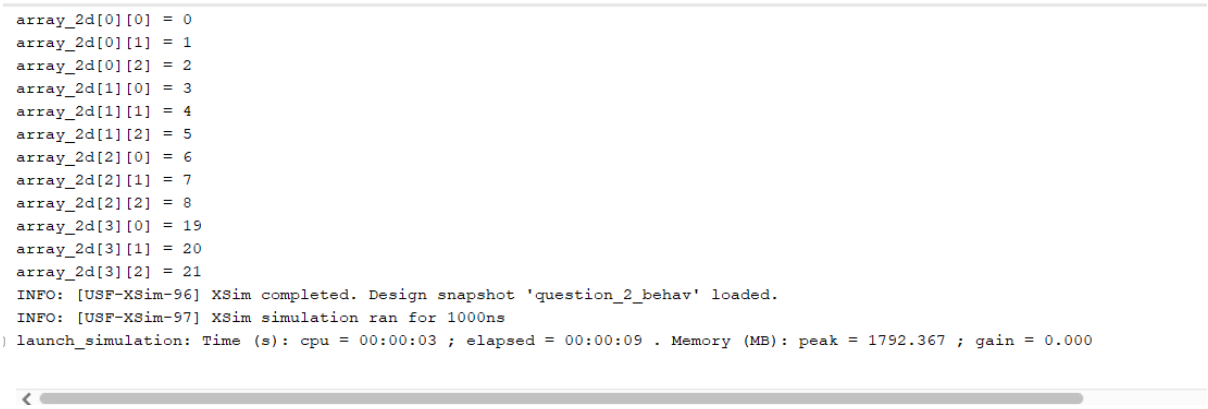
```
    array_2d[2][1] = 7;
    array_2d[2][2] = 8;

    array_2d[3][0] = 19;
    array_2d[3][1] = 20;
    array_2d[3][2] = 21;

            foreach (array_2d[i, j]) begin
                $display("array_2d[%0d][%0d] = %0d", i, j, array_2d[i][j]);
            end
  end

endmodule
```

Output-Screenshot:



3. Write the System Verilog code to:

i. Declare two bit-type fixed-size array of size 10 which can store 8bits values. Initialize first array with values {2,4,6,8,10,12,14,16,18,20} and second array with values {1,3,5,7,9,11,13,15,17,19}.

ii. Display the values stored in two array declared in step-i.

iii. Copy the contents of first array into second array and compare two arrays and display the result of comparison.

CODE:

```
// Toshiv Kumar(NAME) , 1024060266(Roll Number)
module tb;
  bit [7:0] first_array [10];
  bit [7:0] second_array [10];

  initial begin
    // Initialize first array
    first_array = '{8'd2,  8'd4,  8'd6,  8'd8,  8'd10,
                    8'd12, 8'd14, 8'd16, 8'd18, 8'd20};


    second_array = '{8'd1,  8'd3,  8'd5,  8'd7,  8'd9,
                     8'd11, 8'd13, 8'd15, 8'd17, 8'd19};


    $display("First array:");
    foreach (first_array[i])
      $display("first_array[%0d] = %0d", i, first_array[i]);

    $display("Second array:");
    foreach (second_array[i])
      $display("second_array[%0d] = %0d", i, second_array[i]);

    second_array = first_array;
```

```
    if (first_array == second_array)
      $display("Arrays are equal after copy");
    else
      $display("Arrays are not equal");
  end
endmodule
```

Output-Screenshot:



4. Write the SystemVerilog code for the following requirements.

i. Declare the two integer type dynamic array.

ii. Store the following values in the first dynamic array-{ 3,6,9,12,15,18} and {2,4,6,8,10,12,14} in the second dynamic array.

iii. Print the sum of the elements stored in the arrays.

iv. Insert the contents of the first array into second array (after its last element).

v. Delete the first array and try to print the contents of the first and second array. Write remark if you have any observation.

Code:

// Toshiv Kumar(NAME) , 1024060266(Roll Number)

```
module q4;
  int first_array[];
  int second_array[];
  int sum1;
  int sum2;

  initial begin
    first_array = new[6];
    first_array = '{3, 6, 9, 12, 15, 18};

    second_array = new[7];
    second_array = '{2, 4, 6, 8, 10, 12, 14};

    sum1 = 0;
    sum2 = 0;

    foreach (first_array[i]) sum1 += first_array[i];
    foreach (second_array[i]) sum2 += second_array[i];

    $display("Sum of first_array = %0d", sum1);
    $display("Sum of second_array = %0d", sum2);

    second_array = {second_array, first_array};

    $display("second_array after insert:");
    foreach (second_array[i]) $display("%0d", second_array[i]);

    first_array.delete();
```

```
    $display("first_array after delete, size = %0d", first_array.size());
    $display("second_array after delete of first_array:");
    foreach (second_array[i]) $display("%0d", second_array[i]);
  end
endmodule
```

Output_Screenshot:

I don’t have any particular remark about this. Although, dynamic array feature is quite nice compared to basic static arrays in Verilog.



Learnings from this Experiment:

There are many things that I learned from this experiment. First of all, various data types like integer, int, unsigned int, and bit. I learned about the format specifier feature that is used to declare the minimum width of the output number. I learned how to declare and delete dynamic arrays in system Verilog. At last , I learned an alternative way to initialize for loop-: it is using for each that automatically iterates over all the indexes in an array.